



Hands-On With the Neo4j Graph Platform Session 1



Class Setup



Today Will Be Hands-On!

Options for creating your database

- Create an Aura Database console.neo4j.io (Highly recommended)
- Install Neo4j Desktop from neo4j.com/download

Helpful Links:

Neo4j Developer Materials: <https://neo4j.com/developer/>

Cypher RefCard: <https://neo4j.com/docs/cypher-refcard/current/>

Cypher Manual: <https://neo4j.com/docs/cypher-manual/current/>

Neo4j AuraDB Free Setup

We are going to:

1. Go to console.neo4j.io
2. Sign in & click “Create Instance”
3. Selected “AuraDB Free” database size
4. Give your database a name
5. Make a copy of the generated password - **Save the generated txt!**
6. Wait for 3-5 minutes

Neo4j Desktop Setup

We are going to:

1. Download Neo4j Desktop here: <https://neo4j.com/download-neo4j-now/>
2. Create a new Project
3. In the new Project, Click the 'Add' button and select 'Local DBMS'
4. Create a name and password (at least 8 characters) for the database
5. Select version 5.x
6. Click 'Create'

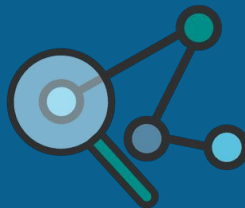
Class Agenda

➤ Labeled Property Graph (LPG) Components 🙌

- What is a graph database? 🙌
- What is Neo4j? 🙌
- Common Graph Use Cases 🙌
- Graph modeling -Part 1 🙌

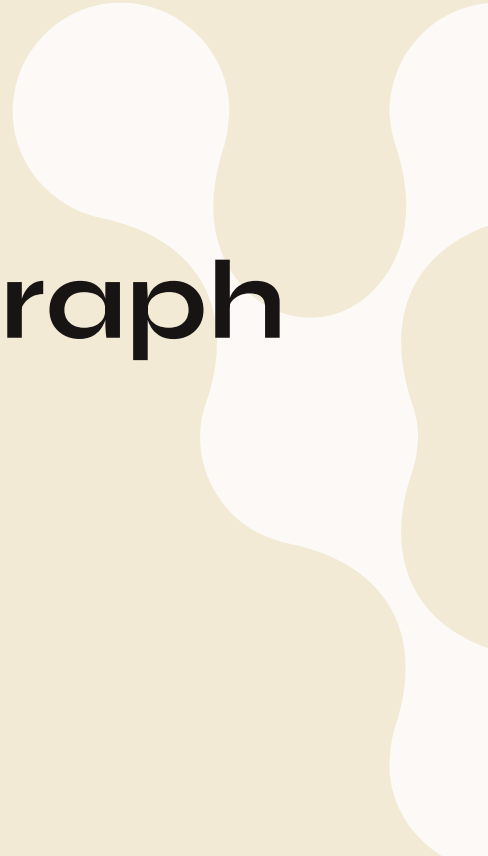
- Graph modeling -Part 2 🙌
- Importing CSV into Neo4j
- Constraints and Indexes
- Intro to Graph Data Science
- Graph Data Science workflows

🙌 Hands-On





Labeled Property Graph Components



Labeled Property Graph Components

Node (Vertex)

- The main data element from which graphs are constructed
A node without relationships is permitted. A relationship without nodes is not.
- Nodes have properties

Relationship (Edge)

- A link between two nodes.
 - Directional
 - May contain metadata; e.g. *weight* or *relationship type*



Neo4j's Property Graph - Nodes

- Represent objects or entities
- Can be *labeled*



Neo4j's Property Graph - Nodes

- Represent objects or entities
- Can be *labeled*
- May have *properties*

name: "Dan"
born: May 29, 1970
twitter: "@dan"



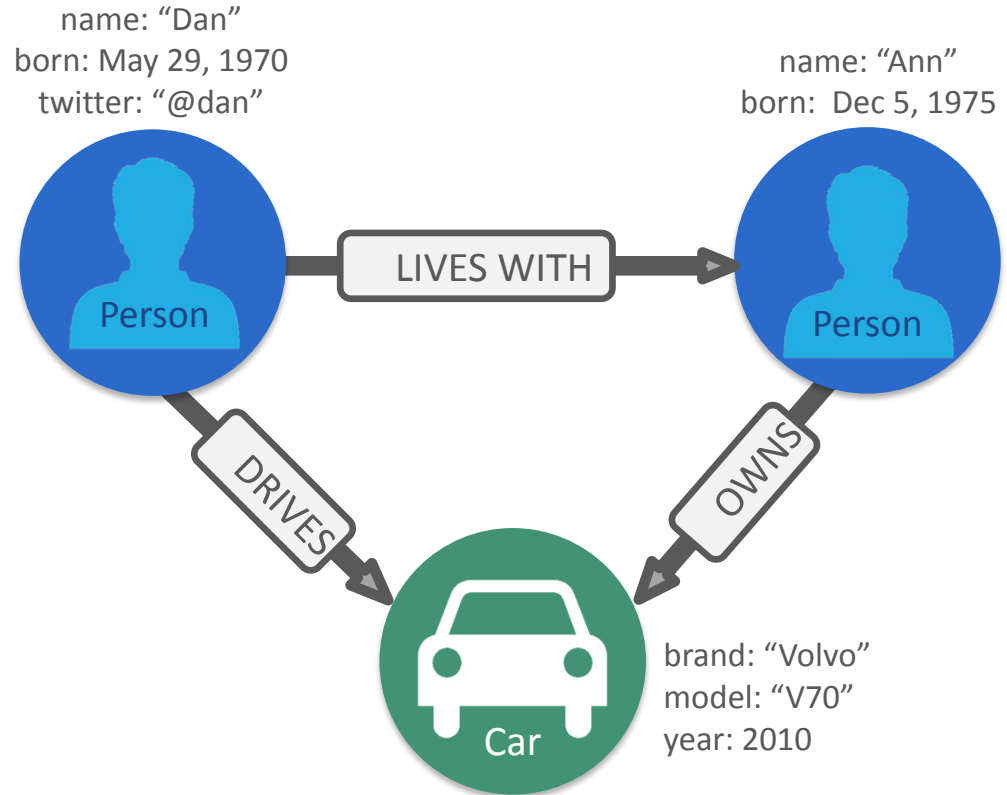
name: "Ann"
born: Dec 5, 1975



brand: "Volvo"
model: "V70"
year: 2010

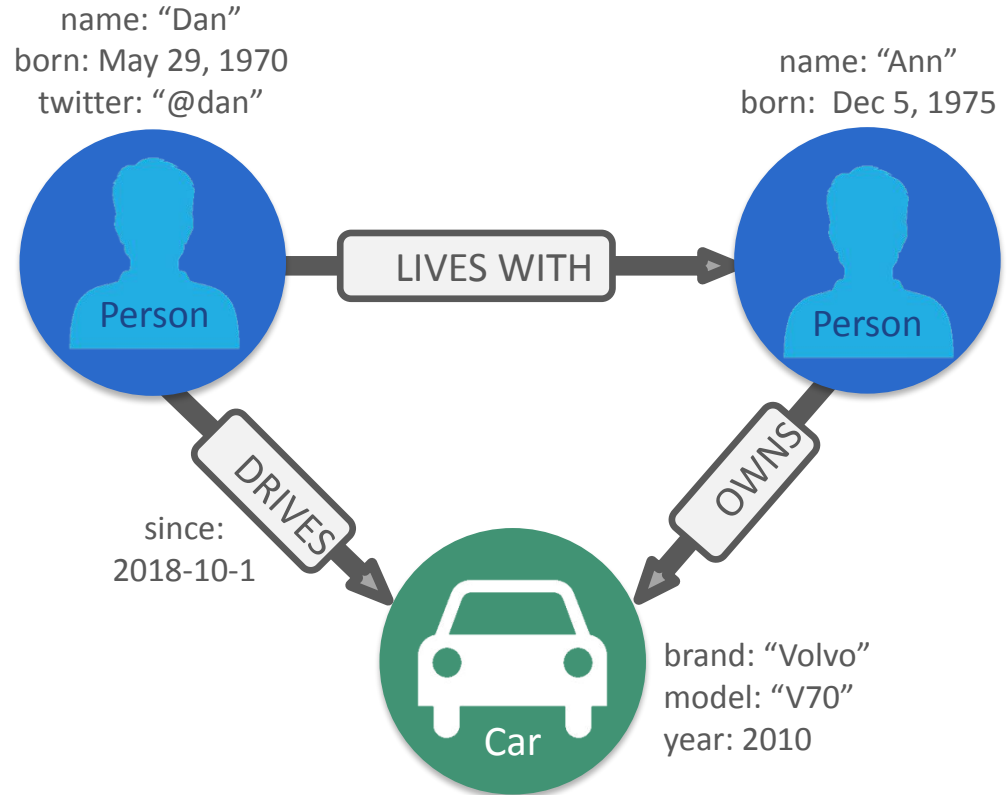
Neo4j Property Graph - Relationships

- Must have a *type*
- Must have a *direction*



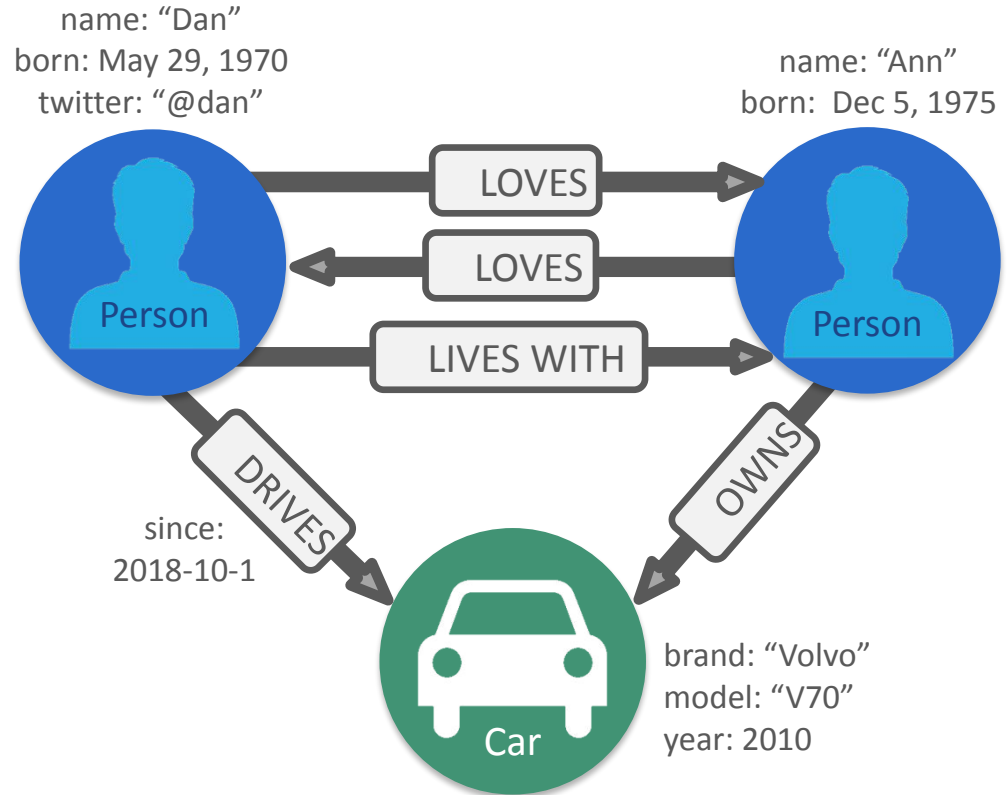
Neo4j Property Graph - Relationships

- Must have a *type*
- Must have a *direction*
- May have *properties*



Neo4j Property Graph - Relationships

- Must have a *type*
- Must have a *direction*
- May have *properties*
- Nodes can share multiple relationships



Neo4j Property Graph

Nodes

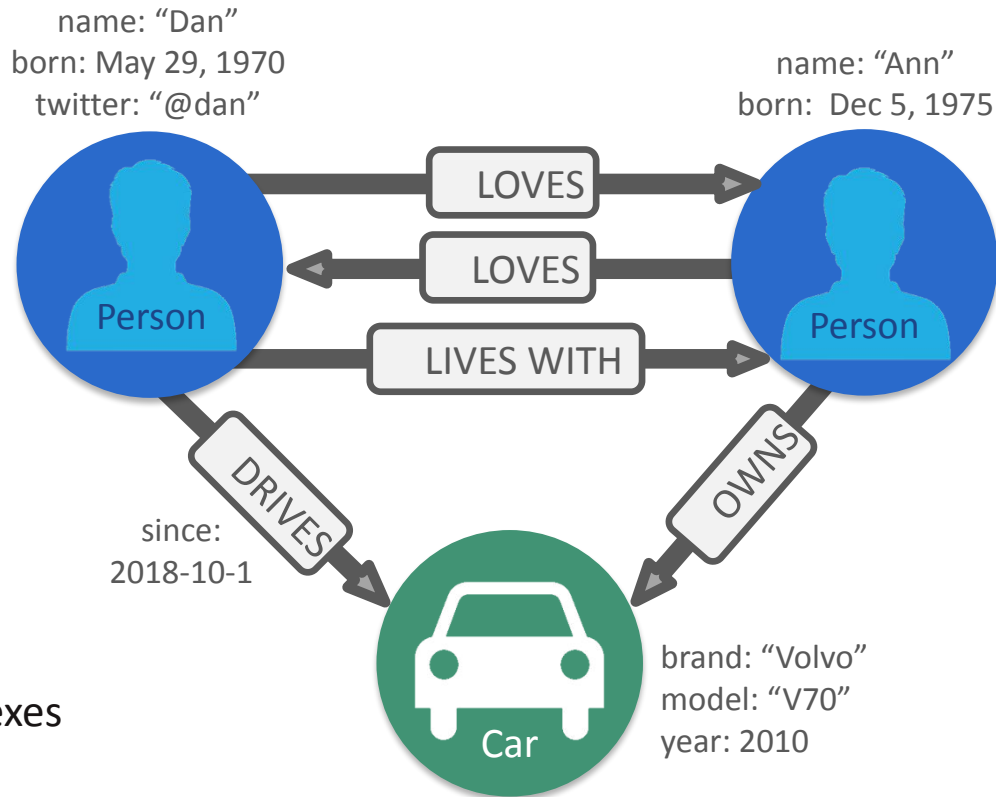
- Can have Labels to classify nodes
- Labels have native indexes

Relationships

- Relate nodes by type and direction

Properties

- Attributes of Nodes & Relationships
- Stored as Name/Value pairs
- Can have indexes and composite indexes
- Visibility security by user/role





Introduction to Cypher

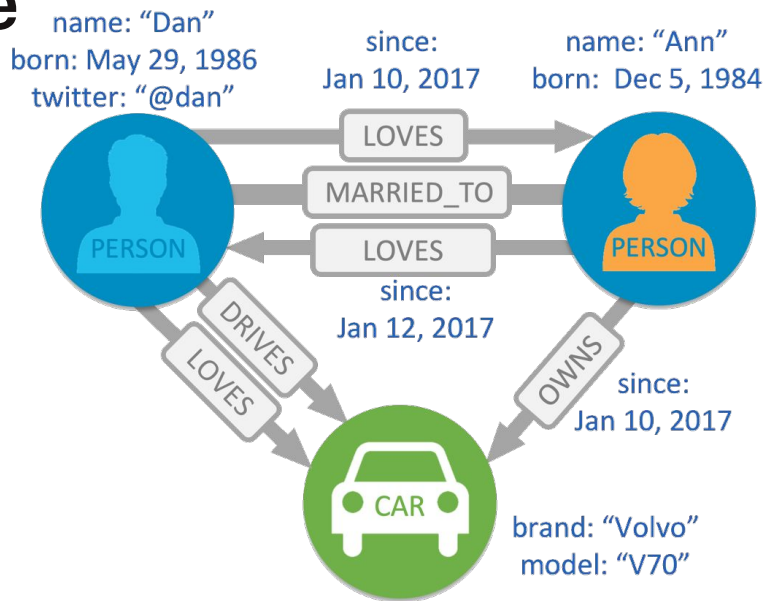


The Cypher Query Language

A pattern matching query language
made for graphs

- Declarative & Expressive
(what to find, not how to find it)
- Pattern Matching

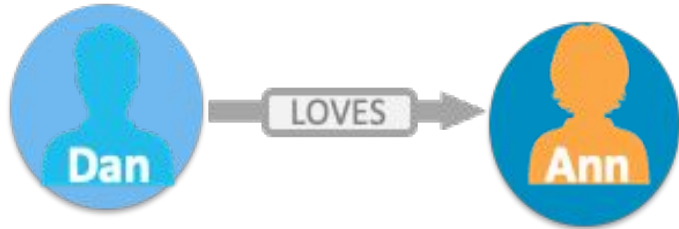
Express Graph Patterns with ASCII ART



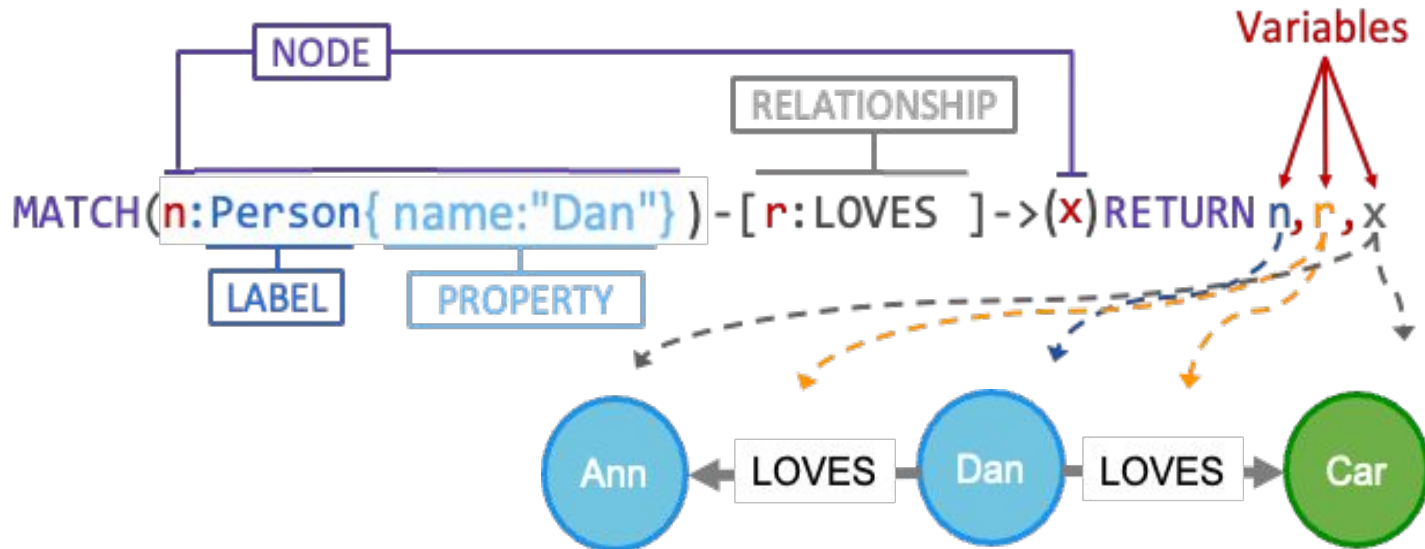
```
(:Person {name: 'Dan'}) -[:LOVES]->(:Person {name: 'Ann'})  
                                -[:OWNS]->(:Car {brand: 'Volvo'})
```

```
(anything) -[:DRIVES]->(something)
```


Cypher Query Language – the **MATCH** clause



MATCH – Return Data



Cypher Overview

Declarative query language

- Focuses on what, not how to retrieve
- E.g. optimizer figures out best approach

Uses **CLAUSE** keywords such as

- MATCH, WHERE, CREATE, UNWIND, WITH, MERGE, ...

Is a dataflow/pipelined language

- Results of each clause in a statement flow into next as materialized

Multi-statement batches supported

- Each statement terminated with “;” (semi-colon)

Foundation for ISO standard query language

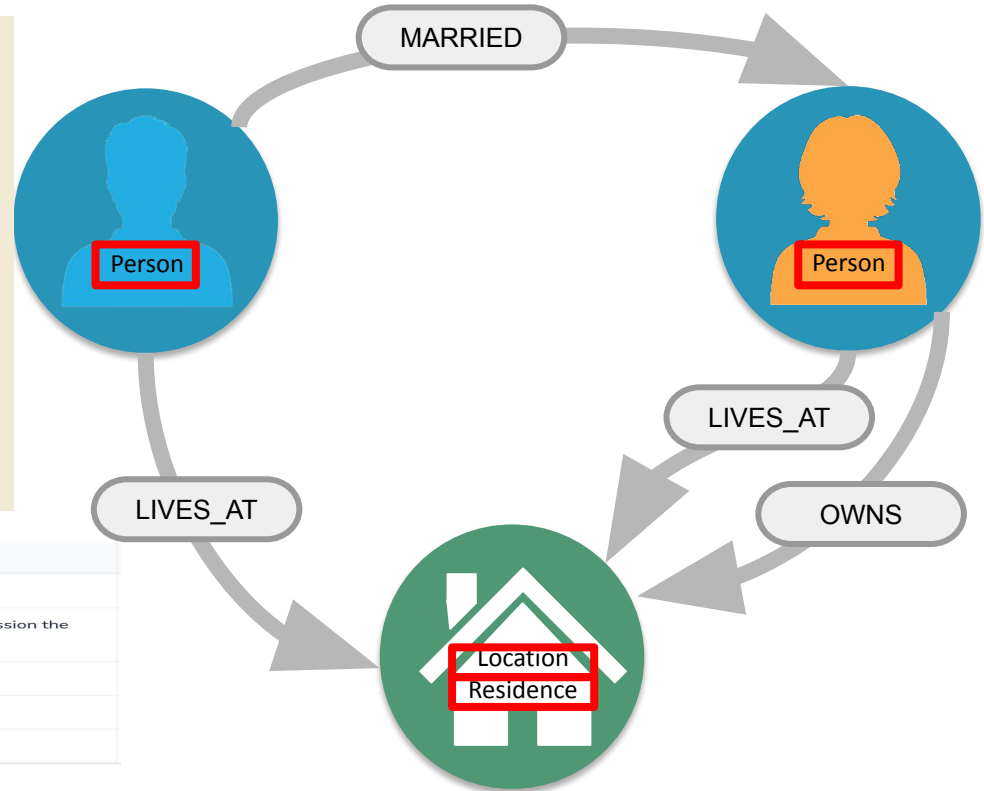
- For graph databases

```
MATCH <pattern> [, <pattern> [, ...]]
[WHERE [NOT] <expression> <op> <expression>
  [[AND | OR | XOR] [NOT] ...]
[OPTIONAL MATCH [WHERE ...]]
[WITH [DISTINCT]
  [WHERE]
  [ORDER BY]
  [LIMIT]]
[MATCH [...]]
[CALL [YIELDING]]
RETURN [DISTINCT]
[ORDER BY ... [ASC|DESC]]
[[SKIP] LIMIT]
[UNION [ALL]]
...
```

Nodes - defined by () - optional labels define node type

```
()  
(p:Person)  
(:Location)  
(l:Location)  
(n:Residence)  
(x:Location|Residence)  
(x:Location & Residence)  
(x:Location &! Residence)
```

Symbol	Description
%	Wildcard. Evaluates to 'true' if the label set is non-empty
()	Contained expression is evaluated before evaluating the outer expression the group is contained in.
!	Negation
&	Conjunction
	Disjunction



Relationships - connect nodes, always directed

```
() -- ()
```

```
() --> ()
```

```
() - [:MARRIED] -> ()
```

```
() - [:LIVES_AT|OWNS] -> ()
```

```
() <-- ()
```

```
() <- [] - ()
```

Relationship types

*(253)

ACTED_IN

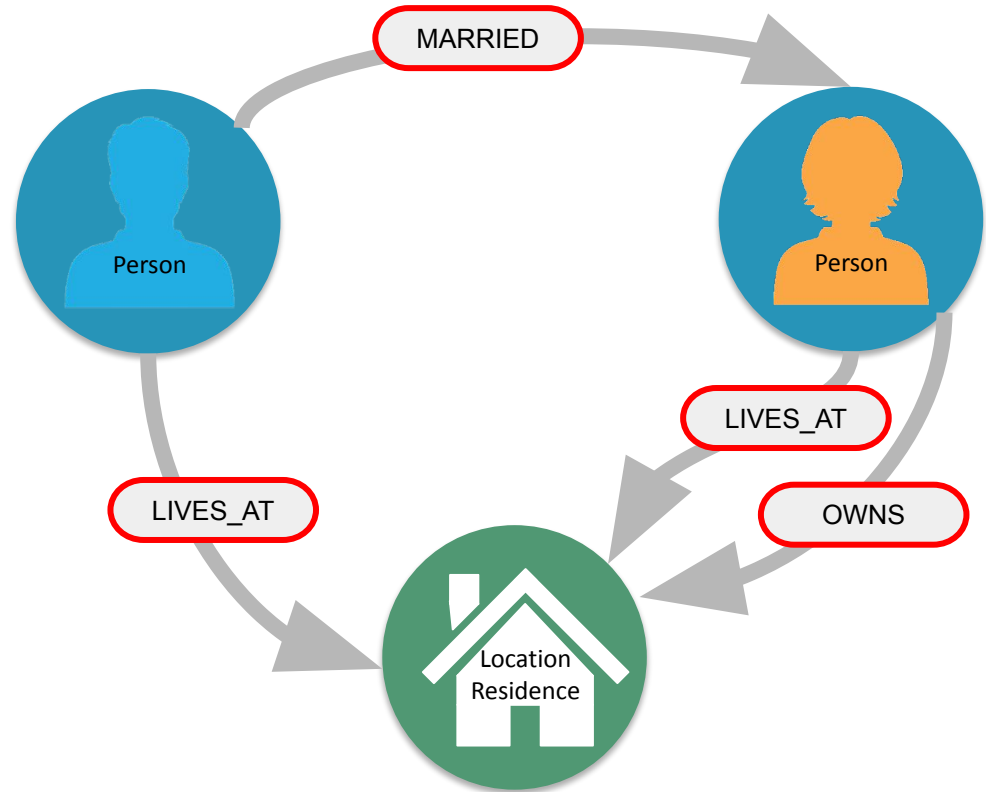
DIRECTED

FOLLOWS

PRODUCED

REVIEWED

WROTE





Hands On Time!

Neo4j Browser Guides

- Interactive guides for learning Neo4j
- Guides provide concepts from data import to graph algorithms
- Find publicly published guides at [GraphGist Portal](#)

- Built-In Browser Guides:
 - :play cypher
 - :play movie-graph
 - :play northwind-graph
 - :play graph-data-science

The screenshot shows the Neo4j Browser interface with the following components and annotations:

- Input browser guide command here:** Points to the command prompt where `:play movie-graph` is entered.
- Click play button to run command:** Points to the blue play button in the top right corner of the editor.
- Click pin to pin tile to top of Browser:** Points to the pin icon in the top right corner of the editor.
- Previous slide:** Points to the left arrow in the bottom navigation bar.
- Next slide:** Points to the right arrow in the bottom navigation bar.

The interface displays the "The Movie Graph" guide, which includes example queries for finding individual nodes, finding movies, and finding people. The queries are as follows:

```
Find the actor named "Tom Hanks"...
```

```
⊙ MATCH (tom {name: "Tom Hanks"}) RETURN tom
```

```
Find the movie with title "Cloud Atlas"...
```

```
⊙ MATCH (cloudAtlas {title: "Cloud Atlas"}) RETURN cloudAtlas
```

```
Find 10 people...
```

```
⊙ MATCH (people:Person) RETURN people.name LIMIT 10
```

```
Find movies released in the 1990s...
```

```
⊙ MATCH (nineties:Movie) WHERE nineties.released >= 1990 AND nineties.released < 2000 RETURN nineties.title
```

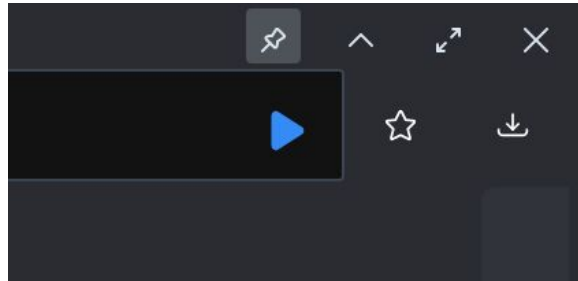
The bottom navigation bar shows the current slide is 3/8, with arrows for navigating between slides.

Now let's get hands on!

Type `:guide hands-on` into your Neo4j Browser command line

```
neo4j$ :guide hands-on
```

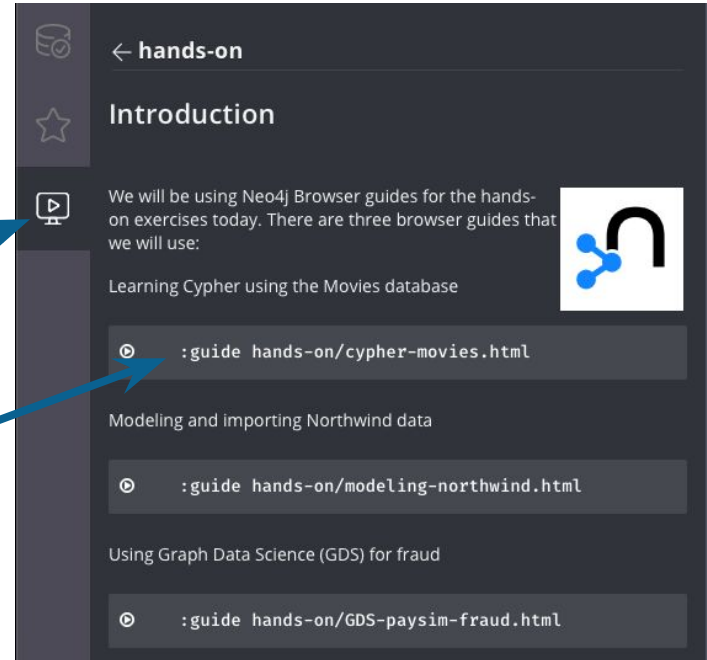
Pin the window -button in the top right corner



1)



2)



Step 1: Load data with multiple create statements

```
CREATE (TheMatrix:Movie {title:'The Matrix', released:1999, tagline:'Welcome to the Real World'})
```

```
CREATE (Keanu:Person {name:'Keanu Reeves', born:1964})
```

```
CREATE (Keanu)-[:ACTED_IN {roles:['Neo']}]>(TheMatrix)
```

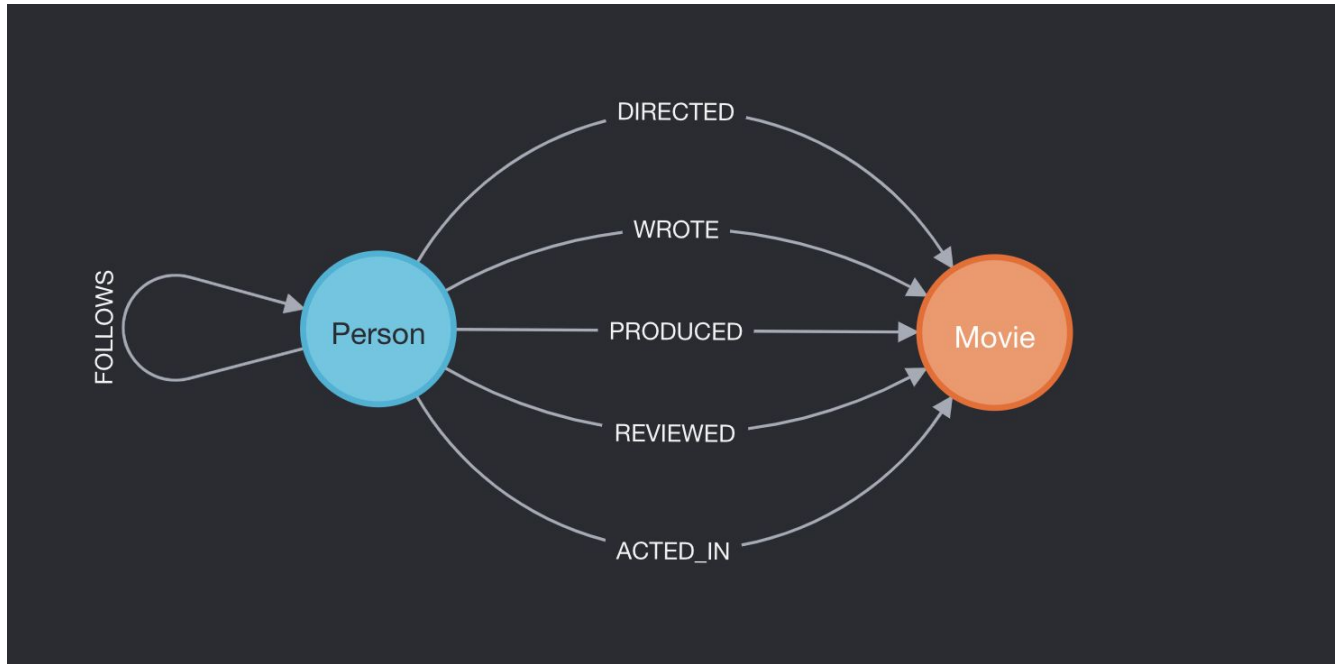
- Above creates a **Movie** node, a **Person** node and an **ACTED_IN** relationship that connects them together
- **TheMatrix** and **Keanu** are the variables that act as placeholders for the created nodes

Note: Only click once! the data will load each time

```
⌚ CREATE (TheMatrix:Movie {title:'The Matrix', released:1999, tagline:'Welcome to the Real World'})
CREATE (Keanu:Person {name:'Keanu Reeves', born:1964})
CREATE (Carrie:Person {name:'Carrie-Anne Moss', born:1967})
CREATE (Laurence:Person {name:'Laurence Fishburne', born:1961})
CREATE (Hugo:Person {name:'Hugo Weaving', born:1960})
CREATE (LillyW:Person {name:'Lilly Wachowski', born:1967})
CREATE (LanaW:Person {name:'Lana Wachowski', born:1965})
CREATE (JoelS:Person {name:'Joel Silver', born:1952})
CREATE
(Keanu)-[:ACTED_IN {roles:['Neo']}]->(TheMatrix),
(Carrie)-[:ACTED_IN {roles:['Trinity']}]->(TheMatrix),
```

Step 2: Investigate the schema

CALL db.schema.visualization





Writing Cypher Queries



Find nodes in the graph

```
MATCH (p:Person) RETURN p;
```

```
MATCH (tom:Person{name: "Tom Hanks"}) RETURN tom;
```

```
MATCH (tom{name: "Tom Hanks"}) RETURN tom;
```

```
MATCH (tom:Person) WHERE tom.name='Tom Hanks' RETURN tom;
```

```
MATCH (nineties:Movie) WHERE nineties.released >= 1990 AND  
nineties.released < 2000 RETURN nineties.title;
```

- MATCH defines the pattern that we are looking for in the graph
- Either use WHERE or map syntax **{name: "Tom Hanks"}** to isolate the particular node or nodes of interest
- RETURN controls the outputs of the query

Using Variables in the graph

```
MATCH (tom {name: "Tom Hanks"}) RETURN tom;  
MATCH (nineties:Movie) WHERE nineties.released >= 1990 AND  
nineties.released < 2000 RETURN nineties.title;  
MATCH (nineties:Movie WHERE nineties.released >= 1990 AND  
nineties.released < 2000) RETURN nineties.title;
```

- A variable in neo4j can be used to alias/represent any node, relationship or value
 - It can contain a list, a map or a literal value
- It must be used anytime you want to
 - Refer to the same node or relationship more than once in the same expression/query
 - E.g., WHERE clause, RETURN clause, WITH clause,
 - Refer to properties of a node or relationship

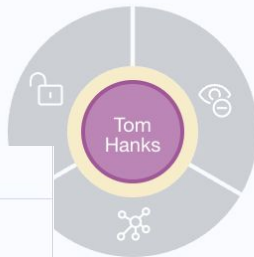
Returning nodes in the graph

```
MATCH (tom {name: "Tom Hanks"}) RETURN tom;
```

```
MATCH (nineties:Movie) WHERE nineties.released >= 1990 AND  
nineties.released < 2000 RETURN nineties.title AS Title, nineties.released AS  
Year ORDER BY Year DESC LIMIT 3
```

- RETURN controls the outputs of the query
 - Return Properties of a node in table format
 - Return Nodes visually

	Title	Year
1	"Snow Falling on Cedars"	1999
2	"The Matrix"	1999
3	"The Green Mile"	1999



Node properties ⓘ

Person

<id>	71	ⓘ
born	1956	ⓘ
name	Tom Hanks	ⓘ

String

Find nodes using filters in the graph

1..MATCH (nineties:Movie{released:1990}) RETURN nineties.title;

2..MATCH (nineties:Movie)

WHERE nineties.released >= 1990 AND nineties.released < 2000

RETURN nineties.title;

**3..MATCH (nineties:Movie WHERE nineties.released >= 1990 AND
nineties.released < 2000) RETURN nineties.title;**

- Key/Value Map in-line in pattern
 - Typical JSON looking map filter with comma separated list of key value pairs
 - Detractors: Can't use inequality expressions or OR logic
- Traditional WHERE clause after pattern
 - Most useful and flexible & Most like SQL
- WHERE clause in-line in pattern
 - Most convenient: add filters when thinking about it vs. trying to remember for WHERE clause later..or any combination of the above!



Hands On Time!

Try it out

:guide hands-on/cypher-movies.html

- What is Cypher?
- Nodes and Relationships
- Labels
- Properties
- Create a Node
- Finding Nodes with Match and Where Clause
- Create a Relationship
- Relationship Types
- Advanced Cypher queries
- Solve the 'Six Degrees from Kevin Bacon'

Cypher Manual

<https://neo4j.com/docs/cypher-manual/current/introduction/>

Cypher Cheat Sheet

<https://neo4j.com/docs/cypher-cheat-sheet/5/auradb-enterprise/>